Day 99: Fine-Tuning a BERT-Style Model on a Real Classification Dataset

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 2 | Complete study guide

Watch: HuggingFace Crash Course - Sentiment Analysis, Model Hub, Fine Tuning (Patrick Loeber) | BERT: Pre-training of Deep Bidirectional Transformers (Paper Explained) (Deep Learning Explainer)

Script: `day99_fine_tuning_bert.py` · every number in this guide is copied by program from `day99_run_output.txt`, the real output of that script.

Plain DistilBERT, which has never seen a sentiment label, fine-tuned on this CPU on small parts of SST-2. On a test half of 436 sentences it reaches 0.844 with 250 items, about 0.851 with 1,000, and 0.869 with 4,000, in 16.3 minutes at most. That is far above a from-scratch bag of words on the same items (0.697 at 1,000) and ahead of frozen pretrained features, but +0.043 behind the ready-made model that was fine-tuned on all 67,349 items. Most of the benefit arrives with the first few hundred labelled examples.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

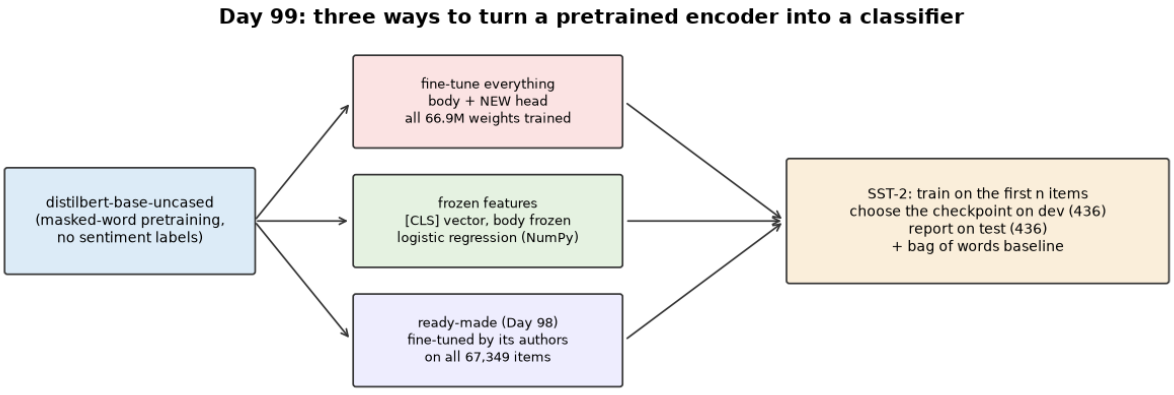
Fine-tuning a pretrained classifier means keeping the pretrained **body**, adding a new, randomly initialised **head** for your labels, and training both a little on your own data at a small learning rate.

The cheaper alternative is **feature extraction**: freeze the body, use its output vectors as fixed features, and train only a simple classifier on them.

REAL-WORLD SCENARIO

Hiring an experienced editor and spending a week teaching them your house style (fine-tuning), against keeping their old habits and adding a checklist on top (feature extraction).

SYSTEM DIAGRAM



Three ways to turn a pretrained encoder into a classifier, all scored on the same test half.

Today's part for Day 104

`day99_exercise.py` is the uncertainty part: `bootstrap_ci` and `paired_bootstrap`, checked on made-up results with a known answer. Its real output:

ACTUAL OUTPUT

```
200 test sets of 436 from a system with true accuracy 0.85: the 95% interval covered it 190 times
    (0.950)
A = B plus 13 fixed mistakes: accuracy A 0.888, B 0.858, they agree on 0.970 of examples
separate intervals: A [0.856, 0.915], B [0.823, 0.890] -> they overlap: True
paired interval for A - B: +0.030 [+0.016, +0.048], A ahead in 1.000 of resamples -> excludes 0:
    True
A against a system with the same accuracy but unrelated errors: +0.000 [-0.046, +0.044] ->
    excludes 0: False
```

A 95% interval covered the truth 0.950 of the time, as it should. And when two systems agree on nearly every example, separate intervals can overlap while the paired interval for the difference excludes 0: the shared examples cancel out.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The setup: a pretrained body and a new head

DEFINITION — start here, no prior knowledge assumed

`from_pretrained(name, num_labels=2)` loads the pretrained body and builds a two-layer head with random weights. Before training the model is therefore no better than a coin toss. Training uses **AdamW**, a linear warm-up then linear decay of the learning rate, and keeps the best checkpoint by dev accuracy.

REAL-WORLD SCENARIO

A new employee who knows the trade but has not yet been told what this company's two filing trays are for.

MATHEMATICS

$$\eta_t = \eta\,(t+1)/w \text{ for } t < w, \quad \text{then } \eta\,(T-t)/(T-w), \quad w = 10\% \text{ of the } T \text{ steps}$$

$$\text{AdamW: } \theta \leftarrow \theta - \eta_t\,(\text{Adam step} + \lambda\theta) \quad \text{(weight decay kept outside the gradient)}$$

SPECIFICATION

Model	distilbert-base-uncased: masked-word pretraining only, no sentiment labels.
Training items	the first n of one fixed shuffle of SST-2's 67,349 training items.
Dev, test	SST-2's 872 validation sentences split once at random, 436 and 436.
Settings	peak rate 5e-5, batches of 16, 2 epochs, 64 tokens, norm clip 1. Not tuned here.

Why this test

Day 98's model was fine-tuned by someone else on everything. The question today is what a learner gets from a little labelled data and one CPU, so the rate stays at the usual value and the amount of data varies.

IMPLEMENTATION

```
def lr_at(step, total):
    warm = max(1, int(WARMUP_FRAC * total))
    return LR * (step + 1) / warm if step < warm else LR * max(0.0, (total - step) / (total -
        warm))
```

ACTUAL OUTPUT

```
distilbert/distilbert-base-uncased: 66,362,880 pretrained weights in the body; 592,130 NEW weights in the head
    (pre_classifier.weight, pre_classifier.bias, classifier.weight, classifier.bias)
dev accuracy with the random head, before training: 0.491 (dev: 436 sentences, 0.532 positive)
data: training items from a fixed shuffle of SST-2's 67,349; dev 436 and test 436 sentences (SST-2 validation split in two,
    test 0.486 positive)
training: AdamW, peak learning rate 5e-05, 10% warm-up then linear decay to 0, batches of 16, 2 epochs, inputs cut at 64
    tokens, gradient norm clipped at 1, weight decay 0.01 (not on biases or LayerNorm); 8 dev checks per run, best dev
    checkpoint kept
Actual output from this lesson's run.
```

What it means

The body brings 66,362,880 pretrained weights; only 592,130 are new. With the random head, dev accuracy is 0.491, chance level for a dev half that is 0.532 positive.

WHY CHOOSE THIS?

Almost all the model's knowledge is ready before the first step.

WHY NOT THIS? (TRADEOFFS)

Every weight still trains, so each step costs as much as full training.

2. Five fine-tuning runs

DEFINITION — start here, no prior knowledge assumed

The same recipe with 250, 1,000 (three seeds) and 4,000 training items, for two epochs each, in five processes at once. Each run checks dev accuracy 8 times and keeps the best checkpoint; test is measured once, on that checkpoint.

REAL-WORLD SCENARIO

Five trainees given different amounts of practice material and the same two read-throughs.

IMPLEMENTATION

```
for epoch in range(EPOCHS):
    order = rng.permutation(n)
    for i in range(0, n, BATCH):
        model.train()
        idx = order[i:i + BATCH]
        enc = tok([Xtr[j] for j in idx], padding=True, truncation=True, max_length=MAX_LEN,
            return_tensors="pt")
        loss = model(**enc, labels=torch.tensor(ytr[idx])).loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        for g in opt.param_groups:
            g["lr"] = lr_at(step, total)
        opt.step(); opt.zero_grad(); step += 1
        if step in check_at:
            acc = (predict(tok, model, Xd) == yd).mean(); curve.append((step, acc))
            if acc > best[0]:
                best = (acc, step, copy.deepcopy(model.state_dict()))
```

ACTUAL OUTPUT

```
 items  seed  steps  best step     dev    test    test 95% interval  minutes
   250     0     32         16   0.846   0.844      [0.807, 0.876]      4.0
  1000     0    126        126   0.842   0.851      [0.817, 0.883]      8.7
  1000     1    126         78   0.858   0.856      [0.821, 0.888]      8.7
  1000     2    126        110   0.872   0.849      [0.814, 0.883]      7.7
  4000     0    500        500   0.888   0.869      [0.835, 0.901]     16.3
1,000 items, three seeds: test 0.851, 0.856, 0.849; spread 0.007
Actual output from this lesson's run.
```

What it means

Only 32 steps on 250 short items take the model from chance to 0.844. Four times more data adds little (0.851), sixteen times more adds about 0.025 (0.869). Three seeds at 1,000 items land within 0.007 of each other on test, although their best checkpoints differ. The 4,000-item run and one 1,000-item run were best at their last step (500 and 126), so they were still improving: more epochs might help (not tested). Rerunning the script reproduced these accuracies exactly.

WHY CHOOSE THIS?

A few hundred labels and minutes on a CPU give a strong classifier.

WHY NOT THIS? (TRADEOFFS)

Each test interval is about seven points wide: 436 sentences cannot resolve small gains.

3. The alternatives on the same test half

DEFINITION — start here, no prior knowledge assumed

Frozen features: the [CLS] vector of unchanged DistilBERT, standardised, with a NumPy logistic regression whose L2 penalty is chosen on dev. **Bag of words**: Day 98's from-scratch model. **Ready-made**: Day 98's model.

REAL-WORLD SCENARIO

Comparing the trained editor with the old-habits-plus-checklist editor and a word-counting script.

MATHEMATICS

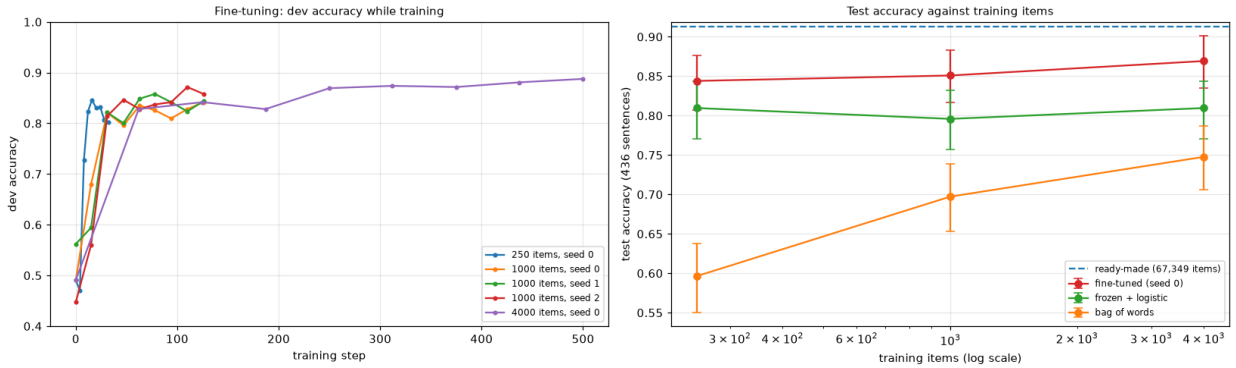
$p = \sigma(w \cdot z + b), \quad \nabla_w = Z^T(p - y)/n + \lambda w, \quad z = \textit{standardised [CLS] vector}$

IMPLEMENTATION

```
def logistic(F, y, Fd, yd, l2s=(1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0, 100.0), steps=500, lr=0.01):
    mu, sd = F.mean(0), F.std(0) + 1e-6
    Z, Zd = (F - mu) / sd, (Fd - mu) / sd
    best, devs = None, {}
    for l2 in l2s:
        w, b = np.zeros(Z.shape[1]), 0.0
        m, v = np.zeros(Z.shape[1] + 1), np.zeros(Z.shape[1] + 1)
        for t in range(1, steps + 1):
            p = 1 / (1 + np.exp(-(Z @ w + b)))
            g = np.r_[Z.T @ (p - y) / len(y) + l2 * w, np.mean(p - y)]
            m = 0.9 * m + 0.1 * g; v = 0.999 * v + 0.001 * g ** 2
            s = lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
            w -= s[:-1]; b -= s[-1]
        acc = np.mean(((Zd @ w + b) > 0) == yd); devs[l2] = acc
        if best is None or acc > best[0]:
            best = (acc, l2, w.copy(), b)
    return best, (mu, sd), devs
```

ACTUAL OUTPUT

```
Frozen DistilBERT: its [CLS] vectors as fixed features, a logistic regression (NumPy) on top, L2 chosen on dev. Bag of words:
    Day 98's from-scratch model. Ready-made: Day 98's model, fine-tuned by its authors on all items.
model                                items    test      95% interval
frozen DistilBERT + logistic (L2 0.1)    250   0.810      [0.771, 0.844]
bag of words (Day 98)                    250   0.596      [0.550, 0.638]
frozen DistilBERT + logistic (L2 10)   1,000   0.796      [0.757, 0.833]
bag of words (Day 98)                  1,000   0.697      [0.654, 0.739]
frozen DistilBERT + logistic (L2 10)   4,000   0.810      [0.771, 0.844]
bag of words (Day 98)                  4,000   0.748      [0.706, 0.787]
fine-tuned DistilBERT (seed 0)           250   0.844      [0.807, 0.876]
fine-tuned DistilBERT (seed 0)         1,000   0.851      [0.817, 0.883]
fine-tuned DistilBERT (seed 0)         4,000   0.869      [0.835, 0.901]
ready-made (Day 98)                   67,349   0.913      [0.883, 0.938]
the frozen baseline's dev accuracy for each L2 penalty (the highest is chosen; the test half is never consulted):
    250 items: 0.0001: 0.761, 0.001: 0.759, 0.01: 0.768, 0.1: 0.787, 1: 0.764, 10: 0.768, 100: 0.764
  1,000 items: 0.0001: 0.768, 0.001: 0.764, 0.01: 0.775, 0.1: 0.778, 1: 0.782, 10: 0.794, 100: 0.534
  4,000 items: 0.0001: 0.752, 0.001: 0.761, 0.01: 0.794, 0.1: 0.807, 1: 0.805, 10: 0.812, 100: 0.534
Actual output from this lesson's run.
```



Left: dev accuracy while training. Right: test accuracy against training items, with 95% intervals.

What it means

Pretraining alone is worth a great deal: with 250 items, frozen features reach 0.810 where the bag of words gets 0.596. But more data barely helps them (0.796 and 0.810), while fine-tuning keeps improving and the ready-made model, with everything, reaches 0.913.

A warning about the L2 choice. The first version of this grid stopped at 0.1, which was chosen, at the grid's edge. Widening it moved the choice to 10: 0.794 at 1,000 items; 100 collapses to the majority class (100: 0.534), so 10 is a true peak on dev. But the frozen model's test accuracy went down with that choice. The dev gaps between the middle penalties are a few sentences out of 436, so this selection is noisy. The dev choice is kept anyway: picking the penalty by its test score would make that score meaningless.

WHY CHOOSE THIS?

Frozen features are cheap, need no gradient through the body, and are hard to overfit.

WHY NOT THIS? (TRADEOFFS)

They cannot adapt: the same vectors whatever the task, so they level off early.

4. Paired comparisons

DEFINITION — start here, no prior knowledge assumed

Two models are scored on the same 436 sentences, so resample the *sentences* and recompute both accuracies each time. The interval for the difference is usually what matters, not whether two separate intervals overlap.

REAL-WORLD SCENARIO

Two runners on the same course on the same day: compare them directly, not each against their own average.

MATHEMATICS

$$d^{(r)} = \text{accuracy}_a(\text{resample } r) - \text{accuracy}_b(\text{resample } r), \quad \text{interval} = \text{middle 95\% of } d$$

IMPLEMENTATION

```
def paired_bootstrap(a, b, n_boot=2000, seed=0):
    rng = np.random.RandomState(seed)
    idx = rng.randint(0, len(a), (n_boot, len(a)))
    d = a[idx].mean(axis=1) - b[idx].mean(axis=1)
    return d.mean(), np.percentile(d, 2.5), np.percentile(d, 97.5), (d > 0).mean()
```

ACTUAL OUTPUT

```
For each pair: mean difference in test accuracy, its 95% paired-bootstrap interval, and the share of resamples in which the
    first model is ahead.
  fine-tuned, 1000          minus frozen + logistic, 1000   +0.055  [+0.021, +0.089]  ahead in 1.000
  fine-tuned, 1000          minus bag of words, 1000        +0.154  [+0.103, +0.206]  ahead in 1.000
  fine-tuned, 4000          minus fine-tuned, 1000          +0.018  [-0.014, +0.050]  ahead in 0.857
  ready-made (Day 98)       minus fine-tuned, 4000          +0.043  [+0.016, +0.071]  ahead in 0.999
  frozen + logistic, 4000   minus bag of words, 4000        +0.062  [+0.016, +0.108]  ahead in 0.994
width of the separate 95% intervals: 0.067 and 0.085; of the paired interval for the difference: 0.103; the two models agree
    on 0.681 of test sentences
Actual output from this lesson's run.
```

What it means

Fine-tuning beats frozen features at 1,000 items (+0.055, interval excludes 0) and the ready-made model beats the 4,000-item fine-tune (+0.043, also excludes 0). Going from 1,000 to 4,000 items is +0.018 with an interval crossing 0: possibly real, not shown by this test set.

The last line is an honest counterpoint to the exercise. Pairing did not narrow this comparison: the paired interval is 0.103 wide, against 0.067 and 0.085 for the separate ones. Those two models agree on only 0.681 of sentences, so their errors are close to independent, and the width of a difference of independent estimates is about the square root of the sum of squared widths. Pairing helps when systems agree on most examples, as near-identical versions of a model do.

WHY CHOOSE THIS?

Asks the right question directly, and needs no assumptions about the shape of the errors.

WHY NOT THIS? (TRADEOFFS)

One test set of 436 sentences; it cannot say how a different sample of reviews would behave.

Interview preparation — technical questions

Q: What does from_pretrained(name, num_labels=2) create?

A: The pretrained body plus a new, random two-class head: 592,130 new weights here.

Q: Why warm up the learning rate?

A: The random head produces large early gradients; a small start avoids a damaging first update.

Q: Why keep the best checkpoint?

A: Dev accuracy moves between checks; the last step need not be the best.

Q: Why exempt biases and LayerNorm from weight decay?

A: It is the usual BERT practice: shrinking them has no regularising purpose.

Q: Frozen features or fine-tuning?

A: Frozen is cheaper; fine-tuning was +0.055 better at 1,000 items here.

Interview preparation — theoretical questions

Q: Why does so little data work?

A: The body already represents language; only the mapping to the labels must be learned.

Q: Why does a paired test sometimes not help?

A: If two models' errors are nearly independent there is little shared variation to cancel.

Q: Why not choose the L2 penalty on test?

A: The test score would stop being an unbiased estimate, even if dev selection is noisy.

Q: What limits accuracy on a CPU?

A: Budget: 16.3 minutes for 4,000 items; the ready-made model trained on all 67,349.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Fine-tuning needs surprisingly little: 250 labelled items and four minutes took plain DistilBERT from chance to 0.844. Frozen features get most of the way and then stop; more data keeps helping fine-tuning, slowly. Compare models with intervals on their differences, and do not read much into gaps of a point or two on 436 sentences.

- Fine-tuned: 0.844, 0.851, 0.869 for 250, 1,000, 4,000 items; seeds at 1,000 within 0.007.
- Frozen + logistic: 0.810, 0.796, 0.810; bag of words: 0.596, 0.697, 0.748; ready-made 0.913.
- Paired: fine-tuned minus frozen +0.055; ready-made minus 4,000-item fine-tune +0.043; 4,000 minus 1,000 +0.018 (crosses 0).

Suggested watch order, if starting from zero

- 1. BERT: Pre-training of Deep Bidirectional Transformers (Paper Explained) (Deep Learning Explainer) — what BERT is and how it was pretrained.
- 2. HuggingFace Crash Course - Sentiment Analysis, Model Hub, Fine Tuning (Patrick Loeber) — the library, including fine-tuning, in practice.

Day 100 preview

- **Named entity recognition.** From one label per sentence to one label per word: a pretrained model on CoNLL-2003, scored by whole entity spans.